ETL (G1)

Unit 2: Data Quality and Batch Data Pipelines

Topic: Batch Data Pipelines and Orchestration with Apache Airflow (1)

Reliable Batch Pipeline Design

From manual ETL Execution to Orchestrated Workflows

Lecturer
Breyner Posso M.Sc.
bposso@uao.edu.co

Learning Objectives of this topic

By the end of this topic, students will:

Explain the principles of reliable batch pipeline design, including task decomposition, dependencies, repeatability, validation gates, failure handling, retries, logging, and monitoring.

Design a batch pipeline as a sequence of clearly defined tasks, identifying inputs, outputs, dependencies, validation points, and failure conditions.

Explain the role of workflow orchestration and distinguish orchestration from the data processing logic executed by individual tasks.

Model and execute batch workflows with Apache Airflow, using DAGs, tasks, dependencies, scheduling, retries, logs, and the Airflow Web UI.

Learning Objectives of this topic

By the end of this topic, students will:

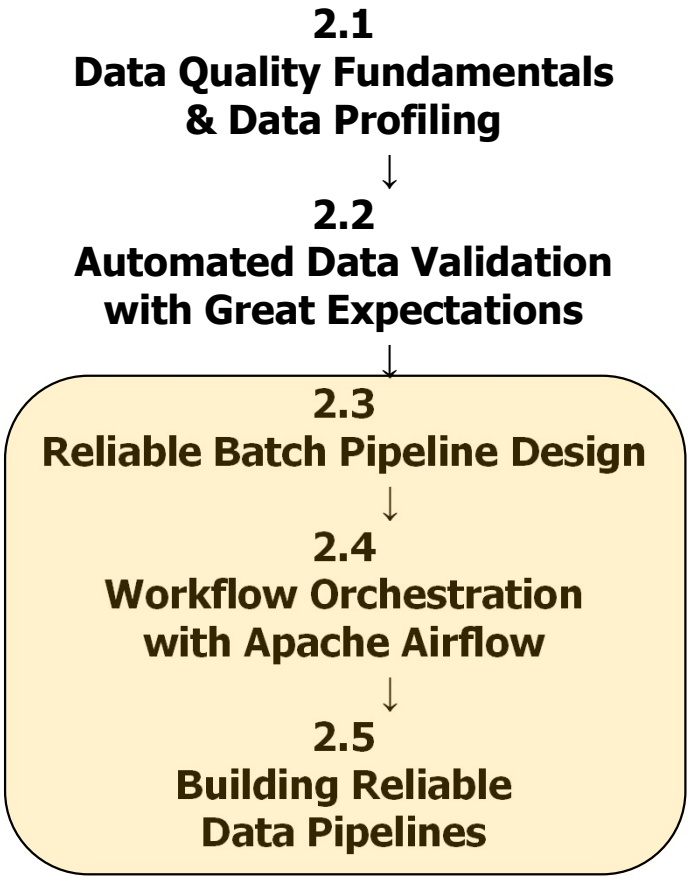
Integrate ETL processing and automated data validation into an orchestrated workflow, using validation results to control downstream execution.

Evaluate pipeline execution and failures using task states, validation results, and logs to determine whether the pipeline should continue, retry, or stop.

Build a reliable batch data pipeline that combines ETL, data quality validation, orchestration, and monitoring.

PART 1 — FROM VALIDATION TO RELIABLE PIPELINES

Unit 2: Where Are We?



The Three Questions

2.3 — DESIGN
How do we design a pipeline that runs reliably and repeatedly?

↓

2.4 — ORCHESTRATE
How do we automate and coordinate the pipeline?

↓

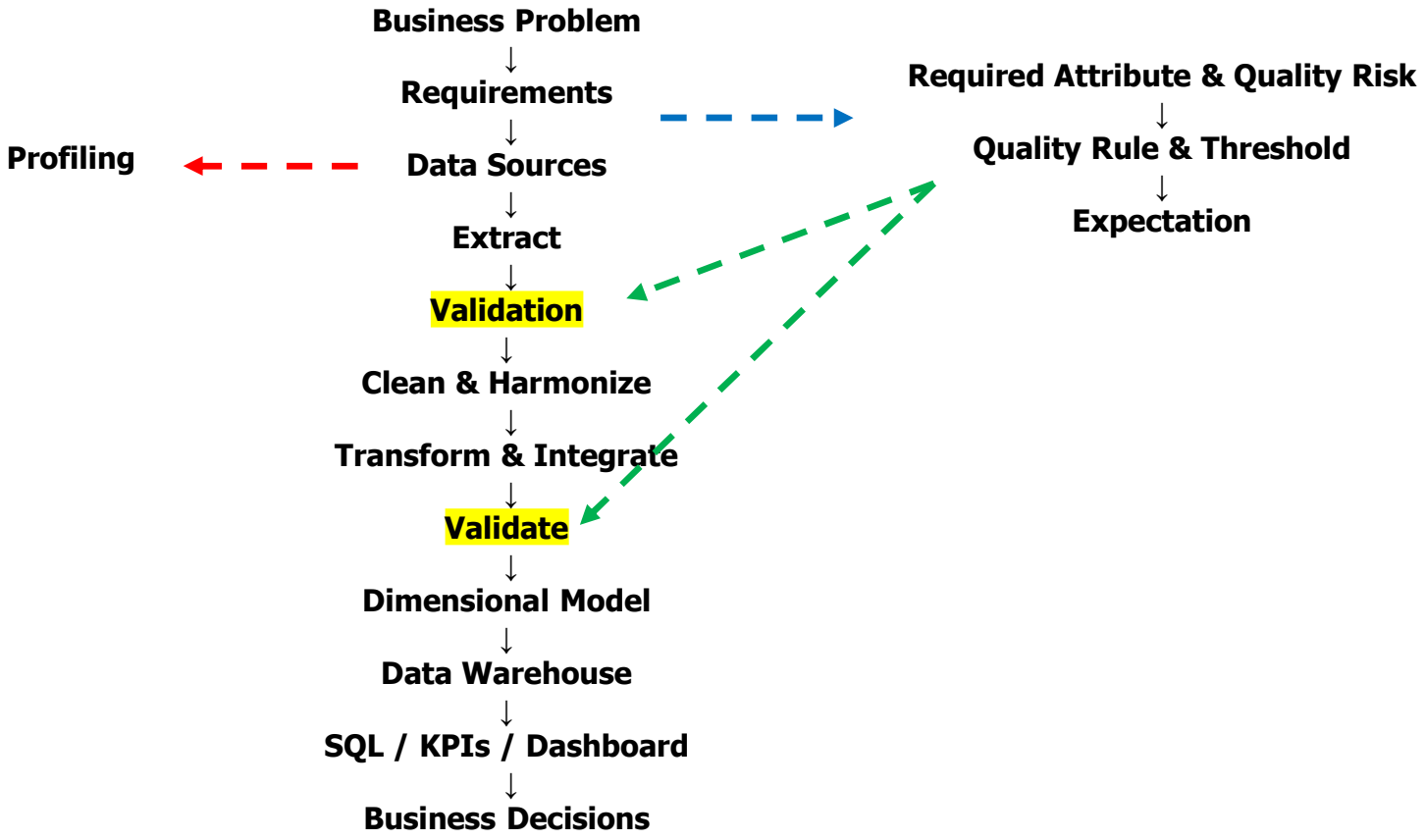
2.5 — INTEGRATE
How do we integrate ETL + validation + orchestration?

In this presentation

Reliable Batch Pipeline Design

Manual ETL
↓
Pipeline Design
↓
Tasks
↓
Dependencies
↓
Validation Gates
↓
Failure Strategy
↓
DAG
↓
First Airflow Execution

What We Already Know



The Missing Problem

We can validate the data…
But who runs the pipeline tomorrow?

- Who starts it?
- In what order?
- What happens if extraction fails?
- What happens if validation fails?
- Should we retry?
- Should transformation continue?
- Where are the logs?
- How do we know what happened?

Manual Execution Does Not Scale

A manually executed ETL process depends on the engineer.

Engineer

↓

run extract.py

↓

check output

↓

run validation.py

↓

run transform.py

↓

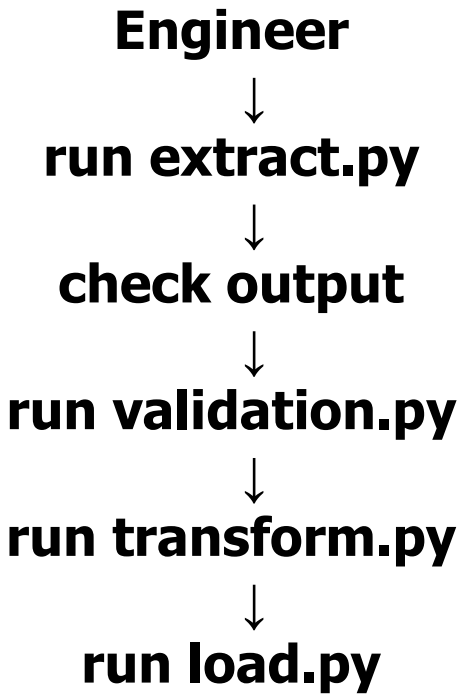
run load.py

Manual Execution Does Not Scale

A manually executed ETL process depends on the engineer.

Possible problems:

- **Repeatability**
- **Dependencies**
- **Failure handling**
- **Monitoring**
- **Traceability**



PART 2 — DESIGNING A RELIABLE BATCH PIPELINE

What should be designed before we automate anything?

What is a Batch Data Pipeline?

Batch Data Pipeline
A workflow that processes a bounded collection of data as a unit, typically at defined execution intervals.

Examples:

- Daily candidate applications
- Nightly transactions
- Weekly customer updates
- Monthly reporting data

Script vs Pipeline

Comparison Table

ETL Script	Data Pipeline
Executes code	Coordinates stages
Usually linear	Explicit dependencies
Limited execution state	Tracks execution state
Manual error handling	Defined failure behavior
Usually one execution	Designed for repeated runs

Script vs Pipeline

Comparison Table

ETL Script	Data Pipeline
Executes code	Coordinates stages
Usually linear	Explicit dependencies
Limited execution state	Tracks execution state
Manual error handling	Defined failure behavior
Usually one execution	Designed for repeated runs

A pipeline is not just code that transforms data. It is an execution system around that code.

Start with Responsibilities

EXTRACT
Obtain source data

VALIDATE RAW
Determine whether incoming data is safe to process

TRANSFORM
Clean, harmonize, integrate, derive

VALIDATE PREPARED
Verify that transformation produced acceptable data

LOAD
Persist trusted data

Where Does Cleaning Happen?

VALIDATION

- detects violations;
- measures quality;
- produces evidence;
- does not automatically decide how to repair data.

TRANSFORMATION

- cleans;
- standardizes;
- harmonizes;
- integrates;
- converts;
- derives.

Where Does Cleaning Happen?

VALIDATION

- detects violations;
- measures quality;
- produces evidence;
- does not automatically decide how to repair data.

TRANSFORMATION

- cleans;
- standardizes;
- harmonizes;
- integrates;
- converts;
- derives.

Validation detects. Transformation applies known engineering rules.

Not Every Error Should Be Cleaned

Example:

Code Challenge Score = 35

Expected range:
$0 \leq \text{score} \leq 10$

Should we change 35 to 10?

Not Every Error Should Be Cleaned

Example:

Code Challenge Score = 35

Expected range:
$0 \leq \text{score} \leq 10$

Should we change 35 to 10?

NO AUTOMATIC CORRECTION

↓

Critical quality failure

↓

Investigate / Stop

Not Every Error Should Be Cleaned

Example:

Seniority = " senior "

What should we do?

Not Every Error Should Be Cleaned

Example:

Seniority = " senior "

What should we do?

- Trim
- normalize case
- standardize category

Seniority = "Senior "

Cleaning requires a justified transformation rule. Validation failure alone does not tell us how to repair the data.

Validation Gates

Data
↓
Extract
↓
VALIDATION GATE
↓
Can the pipeline continue?

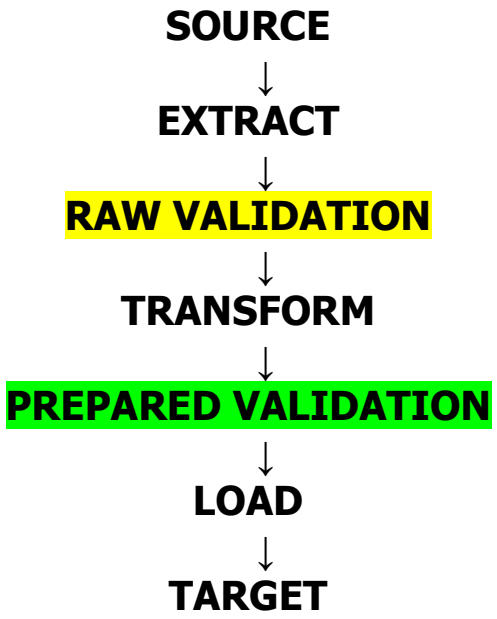
Validation Gates

Data
↓
Extract
↓
VALIDATION GATE
↓
Can the pipeline continue?

- PASS → Continue
- WARNING → Continue + Record / Monitor
- CRITICAL FAILURE → Stop

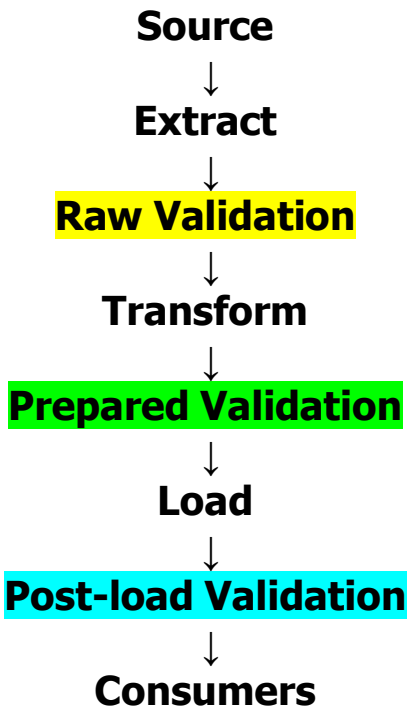
The operational policy determines the response.

A More Reliable ETL Architecture



Quality Gate 1: Is incoming data safe to process?
Quality Gate 2: Did transformation produce acceptable data?

Could We Validate After Load?



Raw: Can we process it?
Prepared: Is it ready to load?
Post-load: Was it loaded correctly?

What Makes the Pipeline Reliable?

Execution Structure
- Tasks
- Dependencies

Data Protection
- Validation gates
- Controlled transformations

Failure Management
- Failure handling
- Retries

Operational Visibility
- Logging
- Monitoring

Repeatability
- Same process, repeated execution

PART 3 — DESIGNING FOR FAILURE

Reliable does not mean that nothing fails.
Reliable means that failures are anticipated, controlled, and observable.

Pipelines Could Fail

- Source file missing
- Database unavailable
- Invalid schema
- Quality rule violated
- Transformation error
- Load failure
- Network interruption

Should every failure be handled in the same way?

Failure Classification

Table

Failure	Possible Response
Temporary network failure	Retry
Database temporarily unavailable	Retry
Missing required column	Stop
Critical quality failure	Stop
Warning quality issue	Continue + log
Programming error	Fail + investigate

Retry is appropriate for transient failures, not for deterministic data problems.

Why Retries?

Transient failure

Attempt 1
↓
Temporary failure
↓
Wait
↓
Attempt 2
↓
Success

Why Retries?

But ...

Score = 35

↓

Retry

↓

Score = 35

↓

Retry

↓

Score = 35

Repeatability

A reliable batch pipeline should produce predictable behavior when executed repeatedly under the same conditions.

- same rules;
- same dependencies;
- same execution logic;
- controlled inputs;
- traceable runs.

Repeatability

A reliable batch pipeline should produce predictable behavior when executed repeatedly under the same conditions.

- same rules;
- same dependencies;
- same execution logic;
- controlled inputs;
- traceable runs.

Ideally, rerunning a task should not create unintended duplicate effects.

Repeatability

Ideally, rerunning a task should not create unintended duplicate effects.

Example:

Load same daily batch twice
≠
Create duplicate records

Logging and Observability

- What ran?
- When?
- With what input?
- Did it succeed?
- Where did it fail?
- Why?
- What happened next?

PART 4 — FROM PIPELINE DESIGN TO A DAG

From Pipeline Design to a DAG

Core Concepts

- Task
- Dependencies
- DAG (Directed Acyclic Graph)
- Operator
- Scheduler
- Web UI

Tasks

A task is a bounded unit of work in the pipeline.

Examples:

- extract data
- validate data
- clean data

Airflow does not process data — it orchestrates tasks.

Tasks

A task is a bounded unit of work in the pipeline.

Examples:

- extract data
- validate data
- clean data

Airflow does not process data — it orchestrates tasks.

Tasks should have a clear responsibility.

Dependencies

Dependencies define the **order in which tasks must be executed**.

Example:

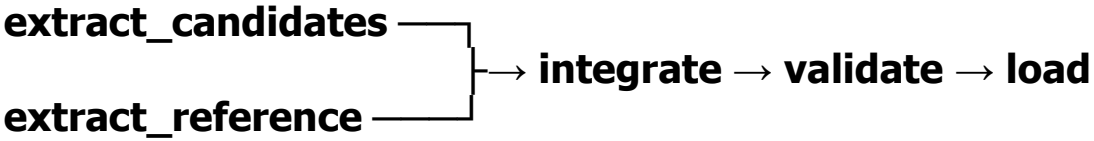
extract
↓
validate_raw
↓
transform
↓
validate_prepared
↓
load

Task B cannot start until the required upstream condition from Task A has been satisfied.

Parallelism

Not all the tasks are linear:

Example:



Some tasks can run independently

DAG (Directed Acyclic Graph)

A DAG defines the structure and dependencies of tasks.

Directed
→ dependencies have direction.

Acyclic
→ execution dependencies cannot form loops.

Example: This estructure is allowed:
A → B → C

DAG (Directed Acyclic Graph)

A DAG defines the structure and dependencies of tasks.

Directed
→ dependencies have direction.

Acyclic
→ execution dependencies cannot form loops.

Example: This estructure is allowed:
A → B → C

Example: This estructure is not allowed:
A → B → C


Our Pipeline as a DAG

A DAG defines the structure and dependencies of tasks.

Example:

extract_candidates
↓
validate_raw
↓
transform_candidates
↓
validate_prepared
↓
load_candidates

PART 5 — INTRODUCTION TO APACHE AIRFLOW

From DAG Design to Workflow Orchestration

What Is Workflow Orchestration?

Workflow orchestration coordinates the execution of tasks according to dependencies, schedules, states, and operational rules.

Pipeline Design

↓

Orchestrator

↓

Coordinated Execution

What Is Apache Airflow?

Apache Airflow is a platform for developing, scheduling, and monitoring batch-oriented workflows.

- Airflow orchestrates the work.
- It does not replace the work.

Web page
https://airflow.apache.org/

Airflow Does Not Replace ETL or GX

ETL CODE
Processes and transforms data.

GREAT EXPECTATIONS
Evaluates data quality.

AIRFLOW
Coordinates when and in what order those operations run.

Airflow Mental Model

Core concepts

DAG
- What is the workflow?

Task
- What unit of work executes?

Dependency
- In what order?

Scheduler
- When can tasks run?

Web UI / Logs
- What happened?

DAG vs DAG Run

DAG

- Definition / blueprint

Example:

- candidate_etl

DAG Run

- One execution of that blueprint

Example:

2026-09-30
2026-10-01
2026-10-02

The DAG defines the workflow. A DAG Run is one execution of that workflow.

Task vs Task Instance

Task
- validate_raw

Task Instance
- validate_raw in DAG Run 2026-09-30

A task instance is a concrete execution of that task.

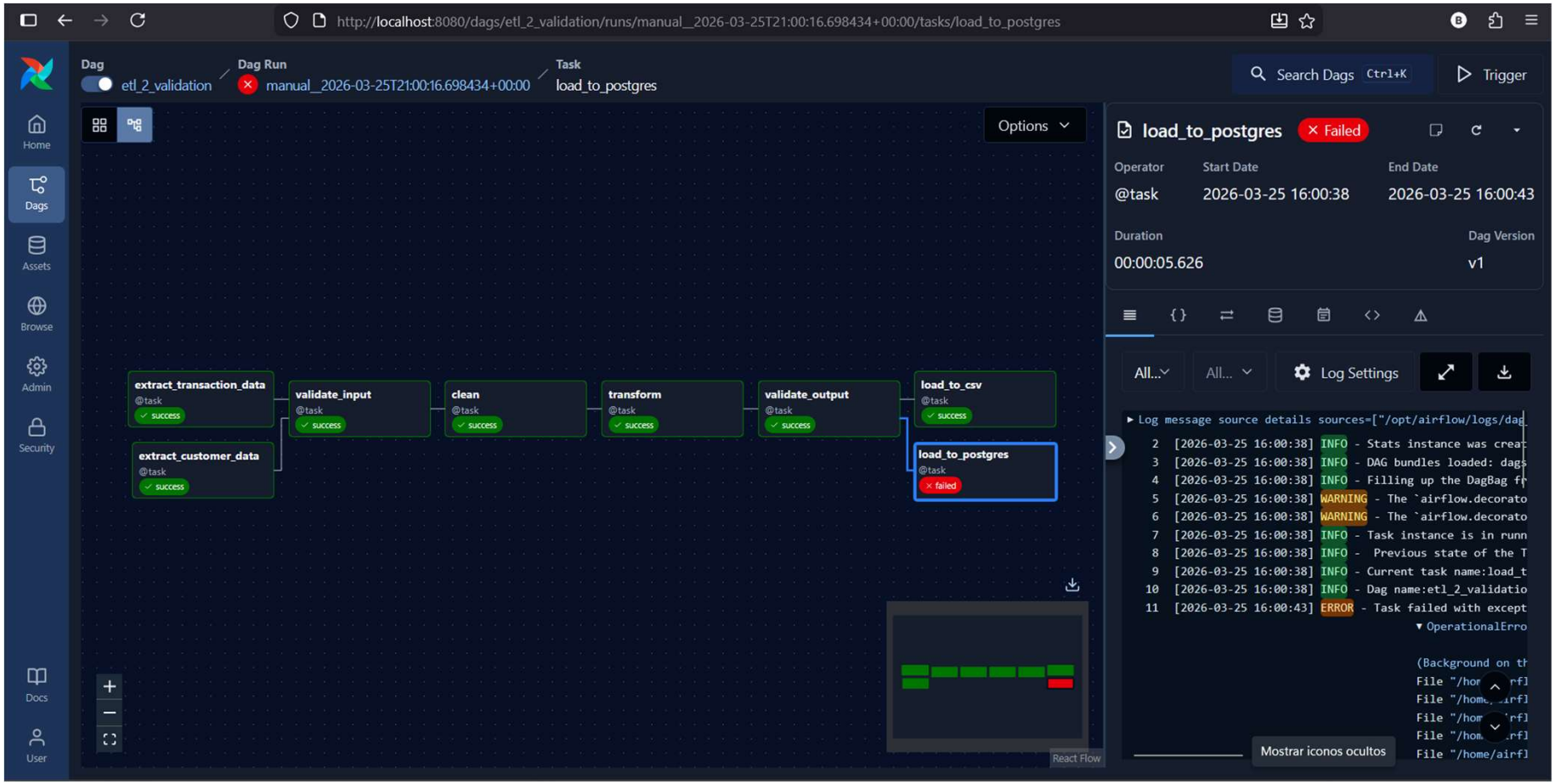
Web UI and Logs

- The Web UI is the graphical interface used to **monitor and manage workflows**.

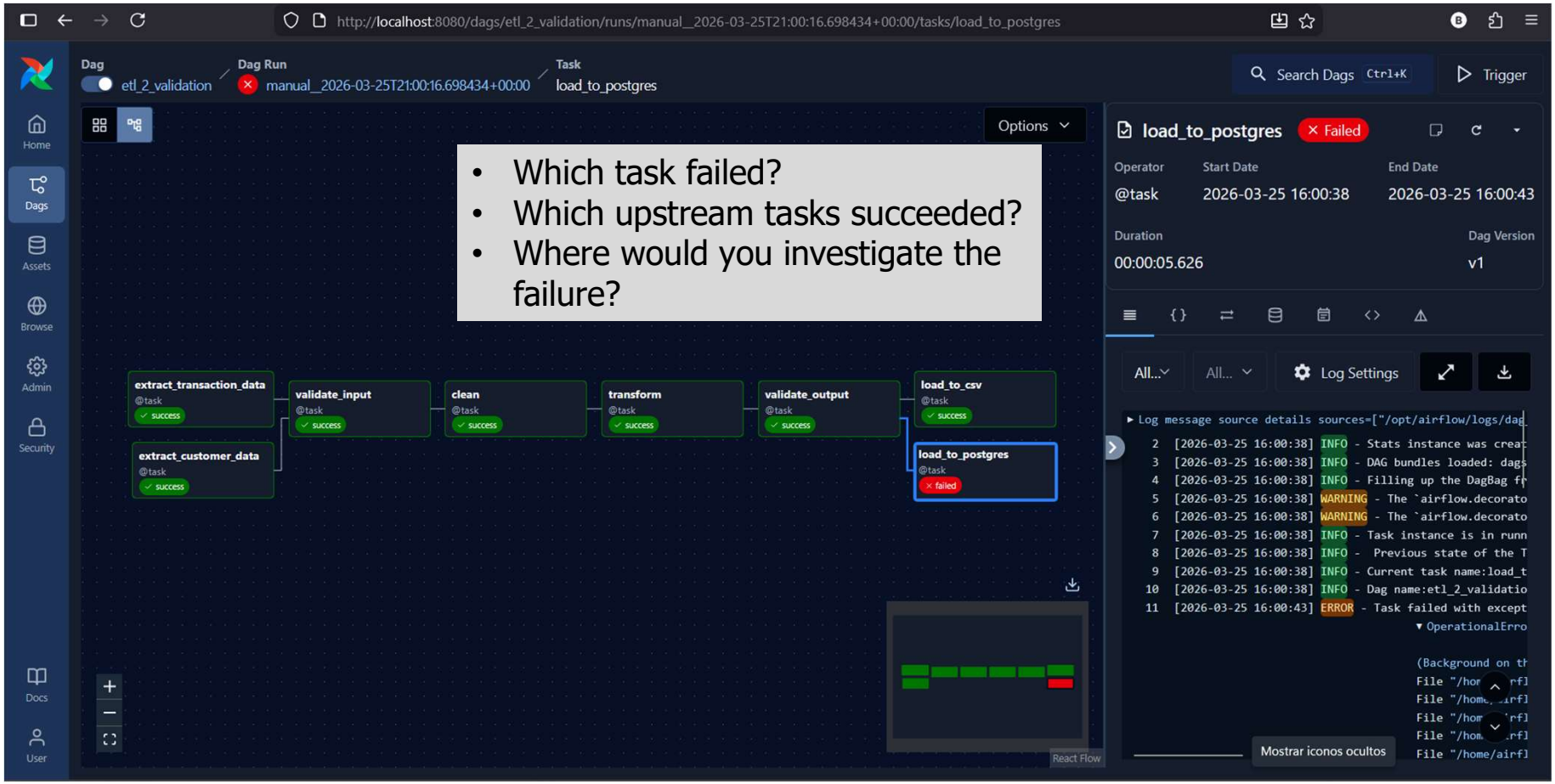
- With the Airflow UI, you can:
  - view DAGs
  - trigger runs manually
  - inspect task status
  - read logs
  - monitor failures and retries
  - visualize dependencies

Why it is useful
It helps data engineers understand **what happened, where it failed, and what to do next**.

Web UI and Logs



Web UI and Logs



Scheduler

- The scheduler is the Airflow component responsible for **deciding when tasks should run**.
- It continuously checks:
  - the DAG definition
  - the schedule
  - task dependencies
  - execution status

What the scheduler does

- triggers DAG runs
- schedules ready tasks
- respects dependencies
- retries failed tasks when configured

Key idea

The scheduler is the **engine that keeps workflows moving**.

Airflow Components Working Together

How Airflow Works as a System

DAG
↓
Scheduler decides when to run tasks
↓
Operators define what each task does
↓
Web UI shows execution status and logs

PART 6 — GUIDED PRACTICE

From Design to Airflow

From Design to Airflow

DESIGN

extract
↓
validate
↓
transform
↓
load

From Design to Airflow

DESIGN	AIRFLOW
extract	@task extract()
↓	
validate	
↓	@task validate()
transform	
↓	
load	@task transform()
	@task load()

First DAG

DESIGN

start
↓
extract_data
↓
validate_data
↓
transform_data
↓
load_data

Objective:
Understand orchestration before integrating real processing logic.

What Should You Observe?

In the practice:

- DAG appears in the UI.
- DAG Run is created.
- Tasks execute according to dependencies.
- Task states change.
- Logs are available.
- A failed task affects downstream execution.

Engineering Questions

In the practice:

- Which object defines the workflow?
- Which elements define execution order?
- What is the difference between the DAG and the DAG Run?
- Where can you inspect task execution?
- What should happen to downstream tasks if validation fails?

What Did We Build Today?

Manual ETL

↓

Reliable Pipeline Design

↓

Tasks

↓

Dependencies

↓

Validation Gates

↓

Failure Strategy

↓

DAG

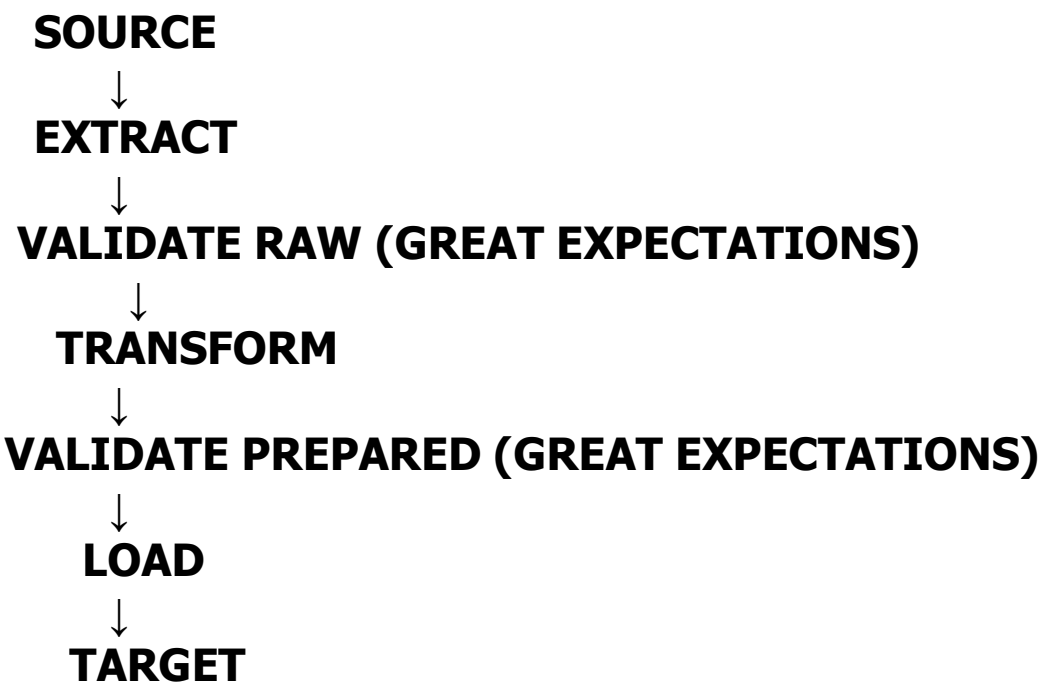
↓

Airflow

Key Takeaways

1. Reliable pipelines are designed before they are automated.
2. Validation and transformation have different responsibilities.
3. Dependencies define how work progresses through the pipeline.
4. Reliability includes controlled failure, retries, logging, and repeatability.
5. Airflow orchestrates pipeline tasks; it does not replace ETL or validation logic.

Next: Building Reliable Data Pipelines



orchestrated by AIRFLOW